Assignment #3 - From Requirements to Design to Implementation

Part A

Brief explanation of the problem.

A patient record system is a crucial element of the overall operational structure of healthcare facilities, containing vital patient information that must be accurate and securely stored. Inappropriate management of this data system, such as the lack of a reliable and structured patient record management system, can lead not only to data inconsistencies, unintentional duplication and alteration of patient records, but also to significant harm to the health of the patients themselves. For this reason, a patient record management system must guarantee the exclusive identification of each patient, data accuracy, and adequate controls over patient data management.

Manual and hand-written patient record systems rely on manual labor to enter any data into the system, causing significant problems like data duplication, accidental patient record modifications, and data loss. It is important for this system to ensure that each patient will have its own unique ID, stored data will have controlled access, and will be protected against unauthorized modifications.

Summary of system purpose.

This Simple Patient Record System is a console-based Python application that allows users to manage patient records using a menu driven system. This system aims to ensure data integrity, streamline access to records, prevent the modification of a Patient ID once it has been assigned, and minimize system errors. This system allows users to:

- Register patients with unique ID
- Retrieve patient records using unique ID
- Update patient names without changing their IDs
- Delete patient records using their Patient ID

In addition, to enforce data integrity, system is based on:

- Patient IDs that are internally generated and cannot be changed after creation
- Exclusive access to all data only using ***PatientRegistry*** class
- **main.py** user interface has no direct access to the patient registry modifications and cannot modify patient ID, which also fulfills REQ-03.

Part B

(REQ-01) The system shall allow the creation of a new patient record with a name and a unique Patient ID.

```
def register_patient(self,name: str) -> str:

    patient_id = "P-" + str( self.id_increment)

    self.patient_registry[patient_id] = {"patient_id": patient_id, "name": name}
    self.id_increment +=1

    return patient_id
```

This requirement is implemented in the ***PatientRegistry*** class using "***register_patient(name: str) -> str***" method that generates a unique ID using "***self.id_increment***" incrementing counter, adds the "***P-***" prefix, returns the unique generated patient ID and stores the patient record in the patient registry dictionary.

(REQ-02) The system shall retrieve a patient's record using their unique Patient ID.

```
def get_patient(self, patient_id:str) -> dict | None:

    if patient_id in self.patient_registry:
        return self.patient_registry[patient_id]

    return None
```

This requirement is implemented in the ***PatientRegistry*** class using "***get_patient(self, patient_id:str) -> dict | None:***" method that checks if given patient ID exists in the patient registry, and returns patient data or ***None*** value if patient ID does not exist. The ***main*** class will display a special message if ***None*** is returned.

(REQ-03) The system shall ensure data integrity by preventing the modification of a Patient ID once it has been assigned.

1. Patient ID is generated only in the ***register_patient()*** method
2. The ***update_patient_name()*** modifies only the patient's name.
3. There is no method in the code that will allow the modification of an existing patient ID.

4. Internal patient registry dictionary exists only inside the ***PatientRegistry*** class and cannot be accessed by user interface.
There is no unique method to implement this requirement – this is a combination of methods that work together to satisfy the third (REQ-3) requirement.

(REQ-04) The system shall allow updating a patient's name using their Patient ID, while keeping the Patient ID unchanged.

```
def update_patient_name(self,patient_id:str, new_name: str) -> dict | None:


    if patient_id in self.patient_registry:
        self.patient_registry[patient_id]["name"] = new_name
        return self.patient_registry[patient_id]


    return None
```

This requirement is implemented in the PatientRegistry class using "***update_patient_name(self,patient_id:str, new_name: str) -> dict | None:***" method that checks if Patient ID exists, and updates only the name of the patient registry field without changing its ID. In addition, this method will return ***None*** value if patient ID does not exist. The ***main*** class will display a special message if ***None*** is returned.

(REQ-05) The system shall allow deleting a patient record using their Patient ID. If the ID does not exist, the system shall display an appropriate error message.

```
def delete_patient(self, patient_id:str) -> bool:
    if patient_id in self.patient_registry:
        self.patient_registry.pop(patient_id)
        return True


    return False
```

This requirement is implemented in the PatientRegistry class using "***delete_patient(patient_id: str) -> bool***" method that checks if Patient ID exists, and deletes a patient record of selected ID from the Internal patient registry dictionary. Additionally, this method returns ***True*** if the deletion was successful, and ***False*** if entered patient ID does not exist. The ***main*** class will display a special message if ***False*** is returned.

Part C

Requirements to Design

When looking over the requirements, the big thing that we noticed was that it all boils down to how to transfer and keep the patient data safely. We designed the system basically around this, and decided that the data and the menu will be kept in different locations (PatientRegistry and Main) so that they won't be together. We also decided to go with a dictionary because it is the most natural in a sense that every patient has a key.

ID Immutability

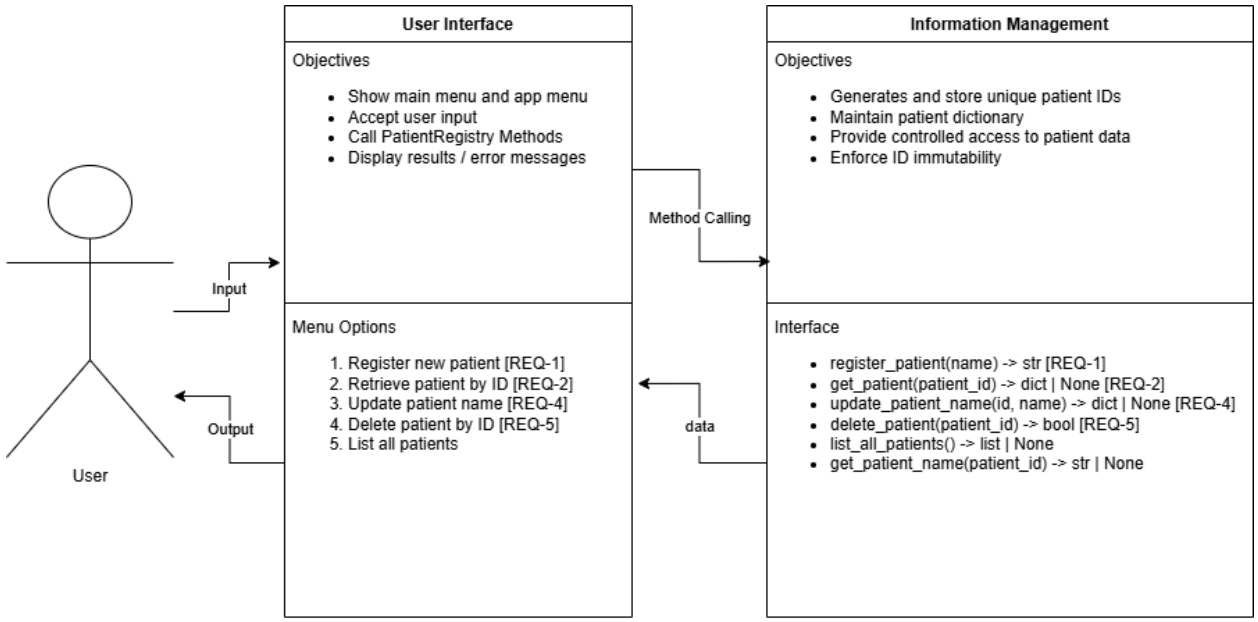
For REQ-3, it is mainly based on being a design rule besides being its own method. The ID is created inside of register_patient(), and then nothing else is done with it. A name can then be assigned or updated using the update_patient_name(), but this doesn't allow for any changes regarding the ID itself. This also protects against attacks since our interface(main) never touches the class that holds them(PatientRegistry).

Error Handling Strategy

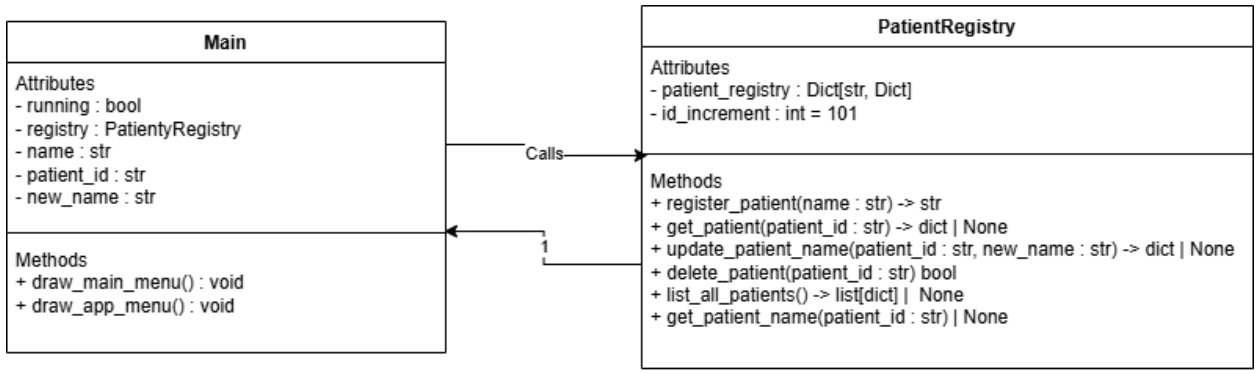
Besides throwing an exception where there is an issue, there are methods that will return either none or false when something goes wrong, for example it will throw a code if the patient ID doesn't exist for some reason. The interface will check the return value and present it to the user, which allows for us not to have a ton of exception blocks to watch for errors.

Diagrams

High-Level Diagram



Class Diagram



Part D

Key Design Decisions

The biggest decision we made design wise would be to keep the PatientRegistry and Main separate from each other for specific reasons. Main will handle what the user sees and types, while the PatientRegistry handles everything to do with the actual data that it has. They can communicate via call and returns, but that is the extent of it. We used a helper method (get_patient_name()) so the man doesn't ever have to go into the actual dictionary to look for a patient ID.

The other big change that we did was to go with none or false for errors besides having exceptions throughout the code. This just makes the code easier to read throughout and doing this instead of exceptions just makes it cleaner overall.

Assumptions Made

The main things that we had to assume were mainly the data focused. For example, we didn't account for if someone were to have the same name inside of the system. If there were two people with the same name, the system wouldn't be able to recognize this if searching by name alone was also allowed. Some others were the data saving between program runs, since this is just a test of building from the requirements and design phase and not building out a complete program.

Part E:

Patient Registration

```
assignment3   master
main.py   PatientRegistry.py
Run   main

Welcome to the Patient Registry Main Menu!

1. Run Application
2. Exit

Enter your choice: 1

#######################################
Application Running...
#######################################

#################################################################
#                                                               #
# a. Register a new patient                                     #
# b. Retrieve patient ID                                        #
# c. Update patient name (ID change NOT permitted)              #
# d. Delete patient ID                                          #
# e. List all patients                                          #
# f. Go back to main menu                                       #
# g. Exit                                                       #
#                                                               #
#################################################################

Enter your choice: a

Enter new patient's name: joh smit

#######################################
Patient Joh Smit registration successful their assigned ID is: P-101
#######################################

Returning to application menu
#################################################################
#                                                               #
# a. Register a new patient                                     #
assignment3 > PatientRegistrySystem > main.py
```

Patient Lookup

```
#                                                            #
##############################################################

Enter your choice: b


Enter patient ID to retrieve patient data (Starts with 'P-'): s

#####################################
Patient does not exist
#####################################

Returning to application menu
##############################################################
#                                                            #
# a. Register a new patient                                  #
# b. Retrieve patient ID                                     #
# c. Update patient name (ID change NOT permitted)           #
# d. Delete patient ID                                       #
# e. List all patients                                       #
# f. Go back to main menu                                    #
# g. Exit                                                    #
#                                                            #
##############################################################

Enter your choice: b


Enter patient ID to retrieve patient data (Starts with 'P-'): p-101

#####################################
Patient P-101's info: {'patient_id': 'P-101', 'name': 'Joh Smit'}
#####################################

Returning to application menu
```

Update Patient Name

```
Enter your choice: c


Enter patient ID to update patient name (Starts with 'P-'): p-101

Are you sure you want to change Joh Smit's patient name? (Y/N): y
Enter patient's updated name: john smith

####################################
Name Changed
Updated Info: {'patient_id': 'P-101', 'name': 'John Smith'}
####################################
Returning to application menu

#############################################################
#                                                           #
# a. Register a new patient                                 #
# b. Retrieve patient ID                                    #
# c. Update patient name (ID change NOT permitted)          #
# d. Delete patient ID                                      #
# e. List all patients                                      #
# f. Go back to main menu                                   #
# g. Exit                                                   #
#                                                           #
#############################################################

Enter your choice: c


Enter patient ID to update patient name (Starts with 'P-'): s


####################################
Patient does not exist
```

Patient Deletion

```
Enter your choice: d


Enter patient ID to delete patient record (Starts with 'P-'): s

####################################
Patient does not exist
####################################

Returning to application menu
##############################################################
#                                                            #
# a. Register a new patient                                  #
# b. Retrieve patient ID                                     #
# c. Update patient name (ID change NOT permitted)           #
# d. Delete patient ID                                       #
# e. List all patients                                       #
# f. Go back to main menu                                    #
# g. Exit                                                    #
#                                                            #
##############################################################

Enter your choice: d


Enter patient ID to delete patient record (Starts with 'P-'): p-101
Patient John Smith about to be deleted
Continue, Y/N?: y

####################################
Patient deleted
####################################
Returning to application menu
##############################################################
```

Patient List

```
Enter your choice: e


#######################################
Patient Registry is empty
#######################################


Returning to Application menu
#################################################################
#                                                               #
# a. Register a new patient                                   #
# b. Retrieve patient ID                                      #
# c. Update patient name (ID change NOT permitted)            #
# d. Delete patient ID                                        #
# e. List all patients                                        #
# f. Go back to main menu                                     #
# g. Exit                                                     #
#                                                               #
#################################################################

Enter your choice:
```

```
assignment3   master
main.py   PatientRegistry.py   temp
Run   main
# e. List all patients                                        #
# f. Go back to main menu                                     #
# g. Exit                                                     #
#                                                               #
#################################################################

Enter your choice: e


#######################################
Patient #: 1
Patient Info: {'patient_id': 'P-101', 'name': 'George Washington'}
#######################################


#######################################
Patient #: 2
Patient Info: {'patient_id': 'P-102', 'name': 'John Adams'}
#######################################


#######################################
Patient #: 3
Patient Info: {'patient_id': 'P-103', 'name': 'Thomas Jefferson'}
#######################################


#######################################
Patient #: 4
Patient Info: {'patient_id': 'P-104', 'name': 'James Maddison'}
#######################################


#######################################
Patient #: 5
assignment3 > patient_registry_system > main.py
```

Part F:

Source Code

---

main.py

```
#Kody Graham
#02/22/2026
#Simple Patient Record System
#Class that will handle all of our menu logic and input validations


from patient_registry import PatientRegistry


class Main:
   def __init__(self):
       self.running = True
       self.registry = PatientRegistry()
       self.name = ""
       self.patient_id = ""
       self.new_name = ""
       self.running = True
       self.all_patient_id = []

       # Entry point for all other functions is draw_main_menu()
       while self.running:
           self.draw_main_menu()


       raise SystemExit(0)

   def draw_app_menu(self):
       app_menu_running = True
       while app_menu_running:

print("#################################################################")
           print("#
#")
           print("# a. Register a new patient
#")
           print("# b. Retrieve patient ID
#")
           print("# c. Update patient name (ID change NOT permitted)
#")
```

```
            print("# d. Delete patient ID
#")
            print("# e. List all patients
#")
            print("# f. Go back to main menu
#")
            print("# g. Exit
#")
            print("#
#")

print("##############################################################")
            print("")


            choice = input("Enter your choice: ")
            print("")
            choices = ["a", "b", "c","d","e","f","g"]
            #Not a fan or if else chains but for simplicity will use one here if
i add a real GUI i will use a click listener instead
            if choice == "a":
                self.name = input("Enter new patient's name: ").title()

                id_generated= self.registry.register_patient(self.name)

                print("")
                print("#####################################")
                print(f"Patient {self.name} registration successful their
assigned ID is: {id_generated}")
                print("#####################################")
                print("")
                print("Returning to application menu")

            elif choice == "b":
                print("")
                self.patient_id = input("Enter patient ID to retrieve patient
data (Starts with 'P-'): ")
                self.patient_id = self.patient_id.upper()

                patient_record= self.registry.get_patient(self.patient_id)

                if patient_record is not None:
                    print("")
                    print("#####################################")
                    print(f"Patient {self.patient_id}'s info: " +
str(patient_record))
                    print("#####################################")
                    print("")
                else:
```

```
                print("")
                print("#####################################")
                print("Patient does not exist")
                print("#####################################")
                print("")

            print("Returning to application menu")


        elif choice == "c":
            print("")
            self.patient_id= input("Enter patient ID to update patient name
(Starts with 'P-'): ")
            self.patient_id = self.patient_id.upper()

            curr_name = self.registry.get_patient_name(self.patient_id)
            print("")

            if curr_name is not None:
                pick= input(f"Are you sure you want to change {curr_name}'s
patient name? (Y/N): ")

                pick = pick.upper()
                if pick == "Y":
                    self.new_name= input("Enter patient's updated name: ")
                    self.new_name = self.new_name.title()
                    patient_exist =
self.registry.update_patient_name(self.patient_id, self.new_name)

                    if patient_exist is not None:
                        print("")
                        print("#####################################")
                        print("Name Changed")
                        print("Updated Info: " + str(patient_exist))
                        print("#####################################")

            else:
                print("")
                print("#####################################")
                print("Patient does not exist")
                print("#####################################")
                print("")
                print("")

            print("Returning to application menu")
            print("")

        elif choice == "d":
            print("")
```

```
                self.patient_id= input("Enter patient ID to delete patient
record (Starts with 'P-'): ")
                self.patient_id = self.patient_id.upper()

                p_name = self.registry.get_patient_name(self.patient_id)
                if p_name is not None:
                    print(f"Patient {p_name} about to be deleted")
                    conformation = input("Continue, Y/N?: ")
                    if conformation.lower() == "y":
                        patient_exist =
self.registry.delete_patient(self.patient_id)
                        print("")
                        print("####################################")
                        print("Patient deleted")
                        print("####################################")

                    else:
                        print("####################################")
                        print("Patient deletion cancelled!")
                        print("####################################")

                        print("")

                else:
                    print("")
                    print("####################################")
                    print("Patient does not exist")
                    print("####################################")
                    print("")

                print("Returning to application menu")


            elif choice == "e":

                all_patients = self.registry.list_all_patients()
                if all_patients is None:
                    print("")
                    print("####################################")
                    print("Patient Registry is empty")
                    print("####################################")
                    print("")

                else:
                    i = 0
                    for record in all_patients:
                        i += 1

                        print("")
```

```
                        print("#####################################")
                        print("Patient #: " + str(i))
                        print("Patient Info: " + str(record))
                        print("#####################################")
                        print("")

                print("")
                print("Returning to Application menu")


            elif choice == "f":
                print("")
                print("Returning to main menu")
                app_menu_running = False

            elif choice == "g":

                self.running = False
                app_menu_running = False

                print("")
                print("")
                print("")
                print("#####################################")
                print("Thank you for using our Simple Patient Record System!")
                print("Goodbye!")

            else:
                print("")
                print("Please enter a valid choice: Single character from a to g
(do not include the . )")
                print("")


    #Simple menu to start may use pygame to generate a true UI menu later
    def draw_main_menu(self):

        print("")
        print("Welcome to the Patient Registry Main Menu!")
        print("")
        print("1. Run Application")
        print("2. Exit")
        print("")

        choice = input("Enter your choice: ")

        if choice == "1":
            print("")
            print("#####################################")
```

```
            print("Application Running...")
            print("#####################################")
            print("")
            self.draw_app_menu()

        elif choice == "2":
            print("")
            print("")
            print("")
            print("#####################################")
            print("Thank you for using our Simple Patient Record System!")
            print("Goodbye!")
            self.running = False

        elif choice != "1" and choice != "2":
            print("")
            print("Please enter a valid choice: 1 or 2")
            print("")

        return

if __name__ == "__main__":
    app = Main()
```

---

patient_registry.py

```
#Kody Graham
#02/24/2026
#Simple Patient Record System
#Class for our Simple Patient Record System that will handle all access to
registry

class PatientRegistry:
    def __init__(self):

        self.patient_registry = {}
        self.id_increment = 101

    # REQ-01
    #Generates a unique ID and stores the patient
    def register_patient(self,name: str) -> str:

        patient_id = "P-" + str( self.id_increment)
```

```
        self.patient_registry[patient_id] = {"patient_id": patient_id, "name":
name}
        self.id_increment +=1

        return patient_id

    # REQ-02
    #Returns patient data or an error if not found
    #Moddified to return None, this is just to further enforce our access to
patient registry only happening in this class file
    def get_patient(self, patient_id:str) -> dict | None:

        if patient_id in self.patient_registry:
            return self.patient_registry[patient_id]

        return None

    # REQ-03
    #NO PATIENT ID CHANGE ALLOWED

    # REQ-04
    #Modified again just to also allow a return of None to keep my patient exist
check within this class
    def update_patient_name(self,patient_id:str, new_name: str) -> dict | None:

        if patient_id in self.patient_registry:
            self.patient_registry[patient_id]["name"] = new_name
            return self.patient_registry[patient_id]

        return None

    # REQ-05
    def delete_patient(self, patient_id:str) -> bool:
        if patient_id in self.patient_registry:
            self.patient_registry.pop(patient_id)
            return True

        return False

    #Again modified to also allow a return of None just to keep all access to
our registry within this class
    def list_all_patients(self) -> list[dict] | None:

        patients = list(self.patient_registry.values())
        if len(patients) == 0:
            return None
        return patients
```

```
    #Extra helpers to keep access of our registry contained in this class will
go here

    #Just to prevent main from having to access our patient registry
    def get_patient_name(self, patient_id:str) -> str | None:
        record = self.get_patient(patient_id)
        if record is None:
            return None
        return record["name"]
```

Part G:

GitHub Repository

https://github.com/KGraham38/SimplePatientRecordSystem.git

Part H:

Daniyar Alimkhanov:

- **Part A**
- **Part B**

Seth Wojick:

- **Part C**
- **Part D**

Kody Graham:

- **Part E** → Handled the code with input from both Daniyar and Seth after we reviewed and extracted the requirements. Screenshots of our system running.

- **Part F** → Added my source code for both classes.

- **Part G** → Created and maintained our GitHub repository for our Simple Patient Record System